Introduction to DL and GenAI Project

1. Abstract

This project focuses on solving an audio classification problem where the goal is to classify noisy mashup audio clips into one of 10 music genres. Due to limited real-world mashup data, a synthetic dataset was generated by combining different audio stems and adding environmental noise.

Three models were developed:

- A custom CRNN (CNN + LSTM)
- A pretrained EfficientNet model
- A transformer-based Audio Spectrogram Transformer (AST)

The models were evaluated using Macro F1 score, and an ensemble approach was used to improve performance. The final system achieved strong validation performance, with AST being the best individual model.

---

2. Introduction

Problem Statement

This is a multi-class classification problem with 10 output classes. The task is to classify audio mashups into 10 genres such as rock, jazz, classical, etc. The challenge arises due to:

- Noisy audio mixtures
- Overlapping sound sources
- Limited labeled data

Project Objective

- Build at least 3 models (as required)
- Improve classification performance using augmentation and ensembling
- Achieve strong Macro F1 score (>0.80 target)

Structure

The report covers dataset creation, preprocessing, model architectures, training strategies, evaluation, and conclusions.

---

3. Dataset & Preprocessing

Dataset Description

The dataset consists of:

Introduction to DL and GenAI Project

- Genre-wise audio stems (drums, vocals, bass, others)
- ESC-50 dataset for environmental noise
- Test mashup audio files

Since real mashups were limited, synthetic data was generated.

Synthetic Data Generation

Each training sample was created by:

- Randomly selecting stems from different songs
- Combining them into a mashup
- Adding noise using controlled SNR

Techniques used:

- Pitch shifting
- Time stretching
- Noise injection (SNR = 10 dB)

This improved generalization and robustness.

The dataset is balanced since equal samples were generated per genre.

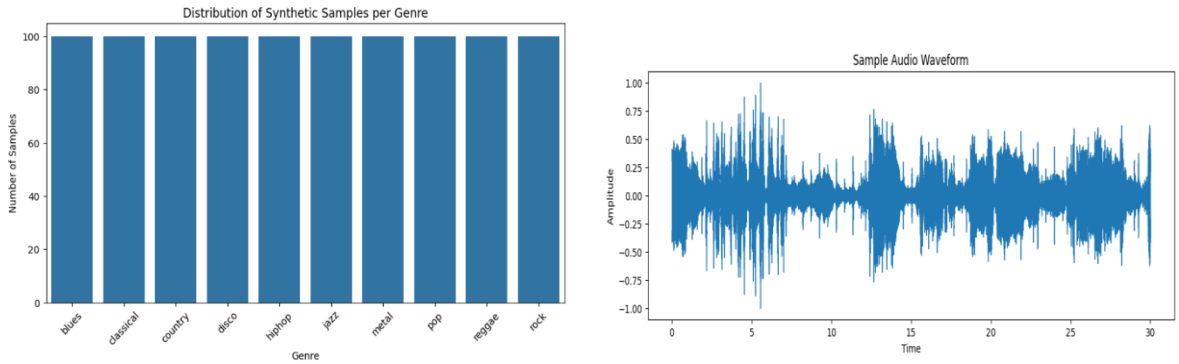

Figure 1: Distribution of samples across genres (balanced dataset) and Sample Waveform

---

Feature Extraction

Audio was converted into **Mel Spectrograms** using:

- Sample rate: 22050 Hz
- n_mels: 128
- n_fft: 2048

- hop_length: 512

Mel spectrogram is used because it represents frequency content in a way aligned with human auditory perception, making it effective for CNN-based learning. Spectrograms were converted to log scale using AmplitudeToDB.

Data Augmentation

- Random cropping (5 seconds)
- SpecAugment:
  - Time masking
  - Frequency masking

These techniques help prevent overfitting.

Pipeline Diagram

Raw Audio
(Stems + Noise)
↓
Synthetic Data Generation
↓
Preprocessing
(Resampling, Mono)
↓
Feature Extraction
(Mel Spectrogram)
↓
Data Augmentation
(SpecAugment)
↓
Model Training
(CRNN / EfficientNet / AST)
↓
Validation (F1 Score)
↓
Ensemble + TTA
↓
Final Prediction

---

4. Modeling & Experimentation

4.1 Model 1: CRNN (Custom Model)

Architecture

- CNN layers extract spatial features
- Residual blocks improve learning stability

- Bidirectional LSTM captures temporal dependencies
- Attention layer focuses on important time steps

Why this model?

Audio is sequential → CNN alone is not enough
LSTM captures temporal structure → improves performance

Architecture Diagram

```
Input
(1 × 128 × 128 Spectrogram)
↓
Conv2D + BatchNorm + ReLU
↓
Residual Block
↓
MaxPool
↓
Conv Block (repeat)
↓
Flatten Time Axis
↓
BiLSTM
(Temporal Learning)
↓
Attention Layer
↓
Fully Connected
↓
Output (10 Genres)
```

4.2 Model 2: EfficientNet (Pretrained CNN)

Architecture

- EfficientNet-B0 adapted for spectrogram input
- Pretrained on ImageNet

Why this model?

- Transfer learning improves performance
- Faster convergence
- Strong baseline for comparison

Limitation: CNN captures local features but lacks global context.

Introduction to DL and GenAI Project

4.3 Model 3: AST (Audio Transformer)

Architecture

- Transformer-based model using self-attention
- Processes raw waveform via feature extractor

Why this model?

- Captures global dependencies
- Handles complex audio patterns better than CNN

This turned out to be the best-performing model.

---

5. Training Details

- Loss Function: CrossEntropy with label smoothing
- Optimizer: AdamW
- Scheduler: Cosine Annealing
- Metric: Macro F1 Score

Training was logged using Weights & Biases.

---

6. Performance & Comparative Analysis

Validation Performance

Model	Val F1
CRNN	0.51
EfficientNet	0.74
AST	0.81

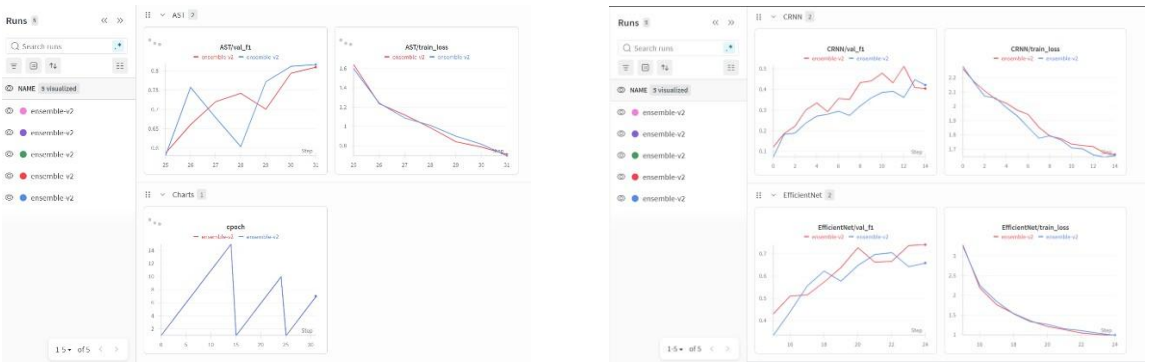

Figure 2 CRNN vs EfficientNet vs AST comparison

Introduction to DL and GenAI Project

Observations

- CRNN struggled due to limited capacity, Lightweight but lower performance
- EfficientNet performed well due to transfer learning, Balanced performance and efficiency
- AST performed best due to global attention, Highest accuracy but computationally expensive

Final Kaggle Score: 0.8386 (Public / Private Leaderboard)

Ensemble Strategy

Final predictions were made using weighted averaging:

- CRNN: 0.15
- EfficientNet: 0.30
- AST: 0.55
- Final Prediction = w1 * CRNN + w2 * EfficientNet + w3 * AST

This improved robustness and final predictions.

Test-Time Augmentation (TTA)

- Multiple crops per audio
- Predictions averaged

This reduced variance and improved stability.

---

7. Conclusion & Future Work

Key Learnings

- Data quality matters more than model complexity
- Transformers outperform CNNs in audio tasks
- Ensemble improves final performance

AST significantly outperformed traditional CNN-based models, confirming the effectiveness of transformer architectures for audio tasks.

Challenges Faced

- Limited real-world data

- High computational cost for AST
- Noise handling in audio

Error Analysis:

Some misclassifications occurred between similar genres such as rock and metal, likely due to overlapping acoustic features. Noise-heavy samples also reduced model confidence.

Future Work

- Try larger transformer models
- Improve synthetic data realism
- Hyperparameter tuning
- Deploy model using Streamlit or Hugging Face

---

8. References

- PyTorch Documentation
- Hugging Face Transformers
- AST Research Paper
- Kaggle Competition Resources